

Testing Perspectives

د. سعيد أبو طراب

محتوى مجاني غير مخصص للبيع التجاري



RB Informatics ; 04/05/2023 هندسة البرمجيات (2)

مقدمة

تحدثنا أنه يوجد لدينا أنواع مختلفة لعملية الـ testing وهي: User, Release, Development وفي هذه المحاضرة سنركز على Development Testing و Unit Testing ومنظورين للاختبارات البرمجية وهما white box و black box. عندما نقوم بفحص أي شيء إما نقوم بفحصه مع معايير بنيته الداخلية أو نقوم بفحصه بناء على وظائفه فقط. فمثلاً للاختبار عمل جهاز الإسقاط (البروجكتر) وظيفياً نقوم بوصله مع الشاشة ونضغط زر تشغيل ليعمل، وممكن أن نقوم بتغيير أبعاد الشاشة من خلاله و كل هذا تم بدون معرفة من كم دارة يتكون وما هي مكوناته الداخلية، ومن هنا ظهر لدينا هذان المنظوران للاختبار:

■ Black Box Testing

فحص البرمجيات وظيفياً من وجهة نظر المستخدم، أي بدون رؤية البنية structure أو المكونات الخاصة بالـ Code حيث يعتمد لتقييم الأداء على مدخلات البرنامج ونتائج الإخراج.

■ White box testing

يكون عكس اختبار الصندوق الأسود حيث يتم اختبار البرمجيات بشكل مفصل و دقيق من خلال دراسة الـ Code الخاص بالبرنامج.

- يمكن أن نفترض أن الـ white box testing يحصل في مرحلة الـ development testing و الـ black box testing يحصل على مستوى الـ release or user testing
- حيث يوجد لدينا مختبر tester يهتم باختبار الوظائف التي علينا تحقيقها بناء على وجهة نظر المستخدم وآخر يهتم باختبار المكونات الداخلية أولاً.

في حال وجود وقت كافي نقوم بعمل white box testing و لكن إذا لم يوجد فمن الضروري عمل black box testing، لماذا؟

لأنه من المهم أن يكون خرج البرنامج حسب الـ user behavior أي أن المختبر سيهتم بتجريب الوظائف التي تهم المستخدم في حال لم يوجد وقت كافي لإجراء نوعي الاختبار. من الممكن أن لا يقوم المستخدم بتجريب جميع الوظائف، على سبيل المثال شاشة التلفاز في المنزل من المهم أن يتم اختبار الوظائف الأساسية لها، ولكن أنت كمستخدم هل قمت بتجريب جميع ميزات الشاشة؟

و لكن في حال وجود تطبيقات أكثر خطورة فهنا يجب علينا ضبط الجودة والأمور من البداية عن طريق تطبيق white box testing و ذلك لأن هذا النوع لا يساعد فقط في التخلص من الخطأ قبل التحيز مما يخفض تكلفة الاكتشاف في وقت لاحق، بل يساعد أيضاً في حال تم اكتشاف خطأ بالقيام بعملية correlation اي معرفة تماماً أين تكمن المشكلة.

Hybrid Testing: ويكون دمج بين black و white حسب الحاجة.

سنقوم بالتركيز على white box testing و بالأخص على مستوى الـ Unit test و ذلك لأن المشكلة الأساسية تكمن في اختيار الـ input المناسبة لاكتشاف الأخطاء.

ك Testing engineer يهمننا:

1. تحديد test cases مناسبة تؤدي إلى اكتشاف الأخطاء.
2. بناء Testing strategy/policy: يجب أن يكون لدينا معيار يحدد لماذا قمنا باختيار هذا الـ test و متى علينا التوقف عن عملية الاختبار (لأن عملية الاختبار بطبيعتها لا تنتهي).

مثال: كود لحل معادلة من الدرجة الثانية:

```
1. double a, b, c, d, x1, x2;
2. d = b*b -4(a*c);
3. if (d >= 0) {
4.     x1 = (-b + sqrt(d)) / 2*a;
5.     x2 = (-b - sqrt(d)) / 2*a;
6.     cout<<x1<<x2;
7. } else
8.     cout<<"no solution";
9. return 0;
```

المدخلات الخاصة بهذا التابع هي a, b, c والمخرجات هي x1, x2.

نولد حالات اختبار مختلفة من الشكل: (1, 1, 1) (مستحيلة الحل) أي a=1, b=1, c=1

(2, 2, 2) (مستحيلة الحل)، (3, 3, 3) (مستحيلة الحل)، (1000, 1000, 1000) (مستحيلة الحل)...

بناء على هذه الاختبارات العشوائية هل نكون قد قمنا بفحص الـ Code بشكل صحيح؟!

فعليا لا، لأننا بجميع الحالات قمنا بالفحص بدون معرفة الحالة البرمجية التي تؤدي الى هذه النتيجة. إذا نظرنا إلى الـ Code و تتبعنا مسار حالات الاختبار سنجد أننا قمنا بفحص نفس المسار بجميعها و التي تؤدي إلى قيمة دلتا سالبة، وباقي المسارات التي تؤدي إلى دلتا أكبر أو تساوي الصفر لم نقم بفحصها. لذا تبين لدينا أنه رغم تطبيقنا لمجموعة من الـ test cases لم نقم بفحص كامل الـ Code، بل قمنا بفحص segment معين منه فقط.

نتيجة: لمعرفة اختيار الـ test cases المناسبة والتي تعطي معامل تغطية محدد يجب علينا أن نكون قادرين على تخيل الـ Code وطريقة سيره.

بملاحظة الـ Code يجب علينا تتبع مسارات كل دخل و لكن هذا سيتطلب الكثير من الوقت و الجهد خاصة في الـ Codes الكبيرة لذلك الأسهل أن نمثل الـ Code ك flow و ذلك باستخدام CFG.

CFG: control flow graph

هنا سندمج الفكرة التصميمية و التنفيذية، حيث كل سطر ضمن الـ Code سنقوم بتمثيله بـ node أيًا كان ما يحويه حيث ما يهمنا هو الـ flow الخاص بالـ Code وليس كل statement وما تمثله، وهذا عكس مخطط flow chart الذي يهتم بالأشكال بناء على محتوى كل statement.

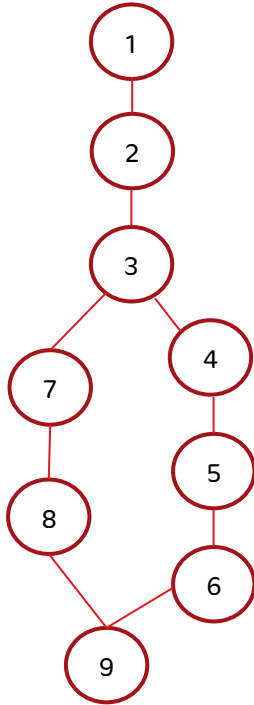
عندما تكون الـ node خارج منها فرعين "2 branches" فهي predicate:

- أول نقطة تسمى input case
- آخر نقطة تسمى output case

نمثل الـ Code السابق كـ CFG بحيث نعطي لكل node رقم السطر

فيكون الشكل:

نلاحظ أن جميع الـ test cases اتبعت المسار (123789).
و منه اصبح لدينا رؤية أفضل لما قمنا باختباره و ما لم نختبره.



معامل التغطية Coverage Criteria

نختار معاملات التغطية لا Testing ضمن الـ policy حسب الـ application.
حتى نقول أن هذه التغطية كافية يجب أن نراعي:

- في حال كان التطبيق احتمالية فشله كبيرة نختار معامل تغطية أدق.
- أما في حال كان تطبيق سهل فنختار معامل تغطية أقل دقة

■ إذا أردنا إجراء اختبارات أكثر علينا اختيار معامل تغطية أدق.

يوجد 3 أنواع لمعاملات التغطية:

1. Statement Coverage
2. Branch Coverage
3. Path Coverage

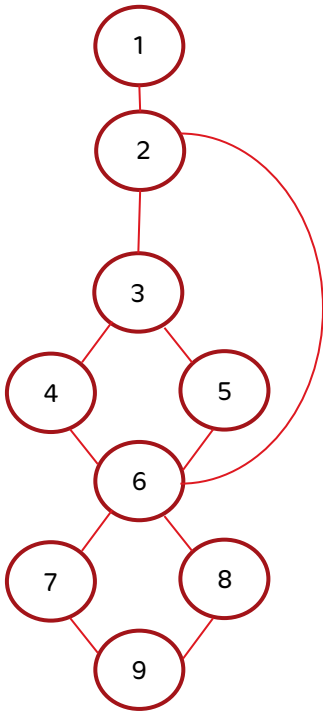
■ Path: هو تسلسل الانتقال من نقطة دخل إلى نقطة الخرج.

■ Test case هي عبارة عن دخل input و الخرج المتوقع expected output و التي تقابل على الخراف path نسميه testing path.

من هو معامل التغطية الأدق أو الذي يولد اختبارات أكثر؟

نلاحظ أنه ليس كل statement لفحصها نحتاج إلى test أو استطيع خلق test case لها لأن ال statement قد لا يكون لها دخل وخرج محددين.

للإجابة على السؤال سنجيب عن الأسئلة التالية بناء على المثال التالي:

كم testing path نحتاج لنحصل على statement coverage؟

نحتاج إلى مسارين لتغطي العقد التسعة: (1234689) و (1235679)

كم testing path نحتاج لنحصل على branch coverage؟

نحتاج إلى 3 مسارات لتغطي الأفرع الثلاثة: 12689 بالإضافة إلى مسارات ال statement coverage.

كم testing path نحتاج لنحصل على path coverage؟

نحتاج إلى 6 مسارات لتغطيتها جميعها وهي:
(1234679) و (1234689) و (1235679) و (1235689) و (12679) و (12689).

- معامل تغطية ال branch يعطي اختبارات أكثر أي هو أدق من ال statement، عندما نحقق branch coverage نكون حققنا ضمناً statement coverage.
- معامل تغطية ال Path هو الأدق ويحقق ضمناً ال branch + statement، وبعدها نقوم بتحديد ال test cases اللازمة للمرور بكل path.
- قد نصل إلى ال infeasible path أي مسار غير قابل للتحقيق ونقوم دائماً بالتحقق من وجوده للاستثناء.

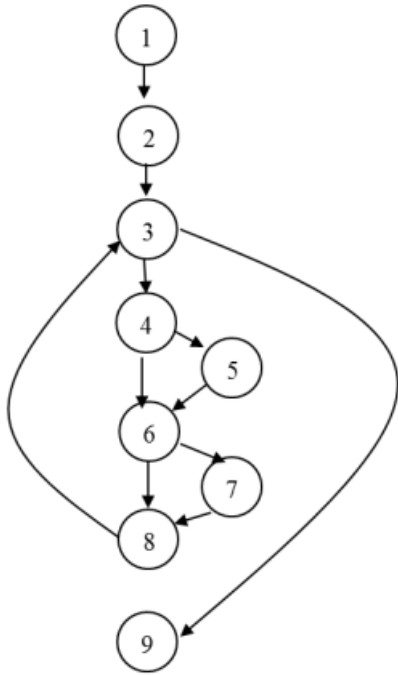
مثال آخر: ليكن لدينا ال code التالي، والمطلوب:

```
1. input (x, y);
2. z=0;
3. while (x+y>0) {
4.     if (x>0)
5.         x=x-1;
6.     if (y>0)
7.         y=y-1;
8.     z=z+x+y;
9. output (z);
```

1. رسم مخطط ال CFG لل Code.
2. إيجاد testing paths for statement coverage.
3. إيجاد testing paths for branch coverage.
4. إيجاد testing paths for path coverage.
5. إيجاد test case لكل path.

الحل:

1. لرسم الغراف نلاحظ أن العقدة 1 تمر على التسلسل مع 2 و تمر على التسلسل مع 3 وتبدأ عملية التفرع عند العقدة 3. نعلم أن الحلقة عبارة عن repeated condition وبالتالي لدينا فرع في حال لم يتحقق الشرط يذهب إلى 9 و يطبع الخرج، أما في حال التحقق سيدخل إلى الحلقة.



لدينا ضمن الحلقة شرطين الأول عند العقدة 4 و الثاني عند العقدة 6.
 في حال تحقق الشرط عند العقدة 4 سيذهب الى 5
 واذا لم يتحقق سيذهب الى 6، وفي حال تحقق الشرط عند العقدة 6
 سيذهب الى 7 و اذا لم يتحقق سيذهب الى 8.
 في العقدة 8 سنعود الى العقدة 3 للتحقق من شرط الحلقة مرة اخرى.
 2. نجد أن مساراً واحداً غطى كل العقد:

Statements = 9

تذكرة: Test Case(input, expected output)

Testing Path (1): 1234567839 => test case (1,1,0)
 أي أننا سنمر في هذا ال Path إذا كان الدخل $x = 1, y = 1$ وسيكون الخرج
 (وهو في مثالنا المتحول z قيمته 0)

3. نحتاج إلى ثلاثة مسارات لتغطية كل الفروع:

Branches = 3

(عند العقدة 3 و 4 و 6)

Testing Path (3): 1239 => test case (1, -1,0)

1234567839 => test case (1,1,0)

12346839

يمكننا نظرياً بمسارين أن نحصل على branch coverage

4. بما انه يوجد لدينا حلقة نستطيع تجريب عدد لا نهائي من الحالات وبالتالي سنواجه مشكلة path explosion اي انفجار بعدد المسارات أي لا نستطيع تحقيق path coverage.

- Path = لا نهائي
- (عدد المسارات التي يمكن اختبارها بغض النظر عن الحلقة) testing path = 5

1239 => test case (0, 0, 0)

1234567839 => test case (1, 1, 0)

123467839 => test case(0,1,0)

123456839 => test case (1,0,0)

12346839

عند إيجاد test cases نلاحظ وجود مسار لا يمكن تحقيقه، حيث أن الذي يحكم المسارات هو الشروط والحلقات، وبالتالي لاختيار المدخلات ننظر لشرط دخول الحلقة والخروج منها.

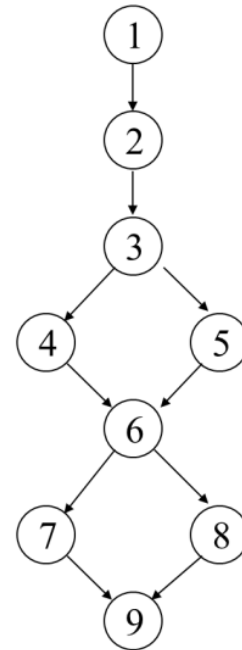
للدخول إلى الحلقة يجب أن يكون مجموع $x + y > 0$ فالمسار 12346839 غير محقق حيث سيقابل كون x ضمن الشرط في العقدة 4 أصغر أو يساوي الصفر و y ضمن الشرط في العقدة 6 أصغر أو يساوي الصفر، وبالتالي مجموعهما أصغر أو يساوي الصفر، أي لم نستطيع دخول الحلقة من البداية، حيث لدخولها يجب أن يكون احد الشرطين في 4 أو 6 محقق و بالتالي هذا مسار مستحيل و لن يتحقق ابداً.

مثال آخر: أوجد Control Flow Graph (CFG) و Test Paths و Test Cases لكل مسار بناء على الكود التالي:

```

1. input (x, y);
2. x=x+1;
3. if (x>0)
4.     x=x+1;
5. else
6.     x=x-1;
7. if (y>0)
8.     y=y+1;
9. else
10.    y=y-1;
11. output (x, y);

```



الـ CFG للـ Code:

Testing paths & Test Cases:

(input: x, y , expected output: x, y)

1234679 => Test Case (1, 1, 3, 2)

1234689 => Test Case (1, -1, 3, -2)

1235679 => Test Case (-1, 1, -2, 2)

1235689 => Test Case (-1, -1, -2, -2)



في حال وجود حلقة سنواجه مشكلة الانفجار بعدد المسارات و لحل هذه المشكلة لدينا ثلاث طرق:

1. تحديد نسبة التغطية المقبول الوصول إليها:

يقوم الـ tester بتحديد الـ coverage criteria ضمن الـ test policy، وتزيد صعوبة هذه العملية مع زيادة تعقيد الكود.

نقول أننا نحقق نسبة تغطية 100% عندما نستطيع إيجاد حالة اختبار واحدة على الأقل لكل مسار. مثال: عندما يكون لدينا 4 مسارات وأوجدنا حالة اختبار واحدة لكل مسار عندها نكون حققنا تغطية بنسبة 100% وفي حال أوجدنا 3 حالات اختبار للأربع مسارات نكون قد حققنا نسبة تغطية 75% وهكذا.

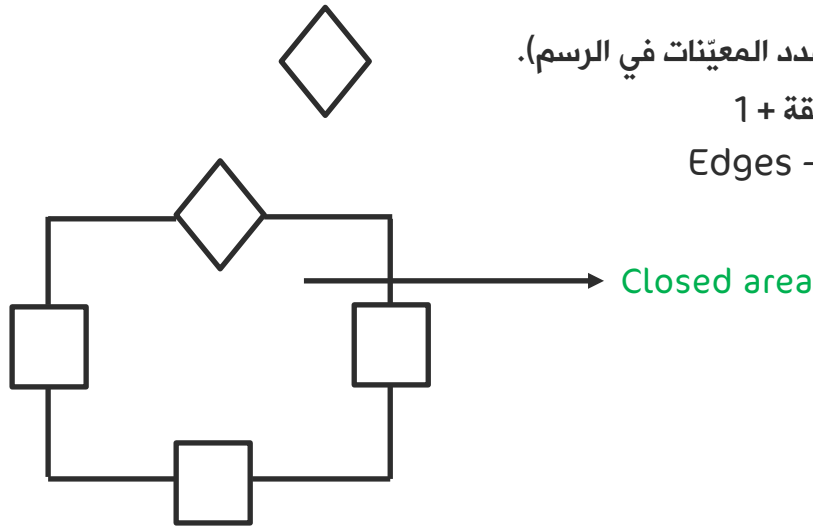
2. التركيز على الـ Basic path:

في حال وجود loops ضمن الاختبار سيكون لدينا عدد لا نهائي من الـ testing paths، وفي هذه الحالة بدلاً من إيجاد الـ path coverage يمكن إيجاد أهم الـ Paths التي يمكن من خلالها إيجاد الـ test case.

Basic path: هو المسار الذي يؤثر على الـ complexity الخاص بالنظام.

ولكي نوجد المسارات الأساسية لدينا قاعدة الـ cyclomatic complexity، وهي تساعدنا بعد إيجاد الـ CFG على إيجاد عدد المسارات الأساسية وذلك بأحد الطرق:

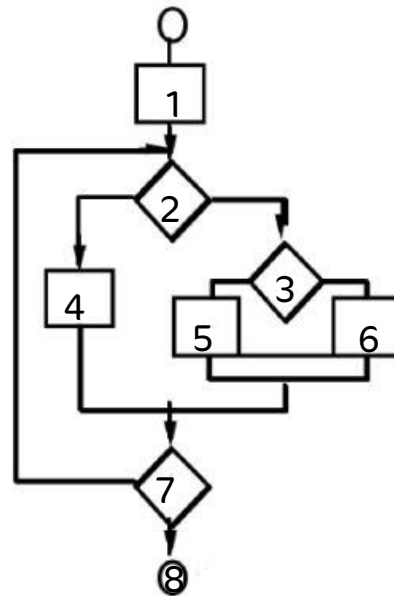
1. عدد الشروط + 1 (عدد المعينات في الرسم).
2. عدد المناطق المغلقة + 1
3. $Edges - Nodes + 2 * p$



مثال:

Path_1 = 1, 2, 3, 6, 7, 8
 Path_2 = 1, 2, 3, 5, 7, 8
 Path_3 = 1, 2, 4, 7, 8
 Path_4 = 1, 2, 4, 7, 2, ..., 7, 8

تذكير : ال Testing path هو مجموعة وصلات بدءاً من نقطة الدخول إلى نقطة الخرج.



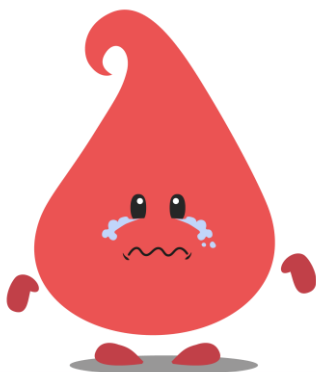
Cyclomatic Complexity = 4

نتيجة: يمكن إيجاد ال Testing path في حالات ال loops عن طريق إيجاد ال Basic paths و التي يمكن أن نوجد عددها عن طريق القاعدة السابقة.

الحلقة في المثال السابق ستسبب مشكلة انفجار المسارات. تمثل الحلقة فعلياً بمسار واحد في قاعدة ال basic paths.

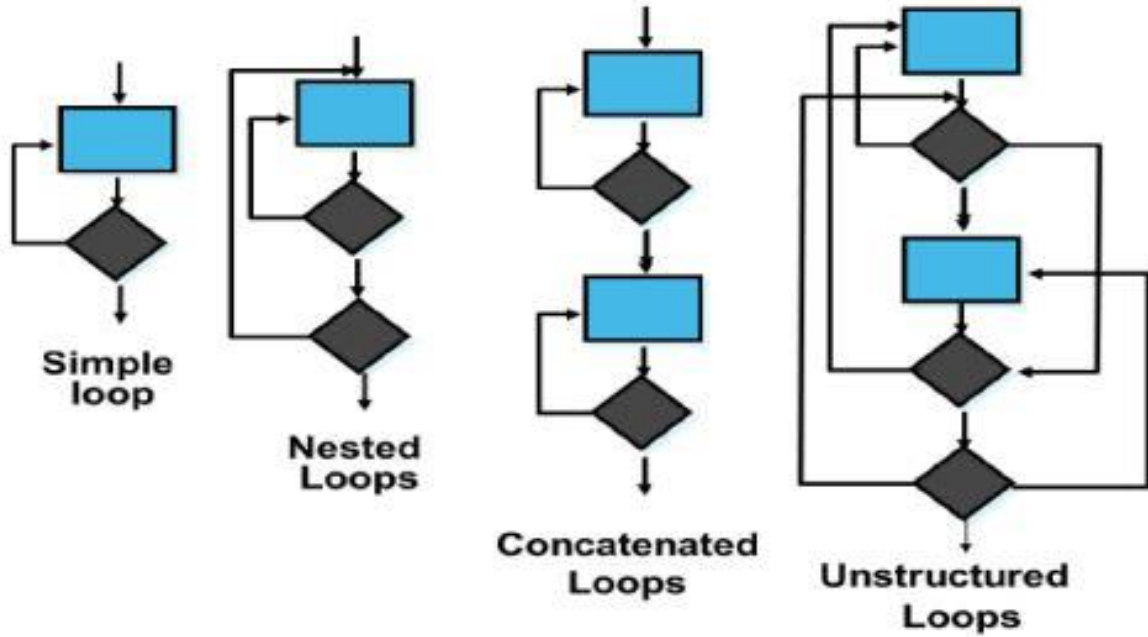
في حال وجود وقت كافٍ لإختبار مسارات الحلقة نقوم ب:

- اختبار نفس المسار بداتاً مختلفة (حيث يحكم الدخول إلى الحلقة قيمة المتغير في شرط الحلقة).
- اختبار أكثر من مسار بداتاً مختلفة.



أنواع الحلقات واختبارها Loops Testing

لدينا أربع أنواع من الحلقات (أيًا كانت while أو for...):



1. simple loop: لها مجال من القيم، لذلك نحاول أن نختبر معظم القيم في المجال، لكن هذا مستحيل مع وجود مجال كبير، فنسعى لاختبار أهم القيم في المجال و تحديد Accepted value و unaccepted value.
- أهم القيم حسب الأولوية: منتصف المجال $[n, m]$ أو أي قيمة مقبولة منه وطرفي المجال (ك n و m)، ونقطتين خارج المجال (ك $n+1, m-1$) مع العلم أننا بهذه الحالة نقوم بفحص أنه يجب عدم دخوله للحلقة).
2. Nested loop: نضمن الدخول للحلقة الداخلية أولاً لاختبارها مع وجود قيم الحلقة الخارجية للـ $\min$ ، و نقوم باختبارها كما في الحلقة البسيطة simple loop تماماً (نختبر أهم القيم من المجال كمنتصف المجال و طرفيه) ثم نركز على اختبار الحلقة الخارجية بنفس الطريقة مع زيادة العداد أو اختبار القيم المهمة و في هذه الحالة لفحص الحلقتين بشكل أكمل نحتاج لـ 10 حالات اختبار 5 لكل حلقة.
3. Concatenated loop: إذا كانت الحلقتان مستقلتان نختبر كل واحدة منهما على حدة كما اختبار الحلقة البسيطة simple loop، و إلا نختبرهما كما اختبار الحلقات المتداخلة nested loop.
4. Unstructured loop: الحلقات الغير منتظمة، لا ينصح باستخدام هذه الحلقات و يفضل تغييرها لنوع آخر، حيث غالباً ما تعاد إلى المطور ليقوم بترتيبها و تحويلها إلى أحد الشكليات simple أو nested.

مثال: سنركز على اختبار الحلقات من نوع simple و nested:

نفحص الحلقة بناءً على الشرط الخاص بها:

```
While (x > 10) {
```

X تنتمي إلى المجال من 11 إلى $+\infty$

من أجل اختبار الحلقة:

- أولاً : نختار قيمة تحقق الحلقة حتماً (و نضمن ذلك تقريباً نصف المجال) حتى نفحص محتوى الحلقة Statement-ك
- ثانياً : بوجود وقت كافي نختبر الحدود لأن الأخطاء تظهر في القيم الحدية (نقطة طرف المجال المحققين للحلقة)
- ثالثاً : بوجود وقت إضافي نختبر قيمتين حديتين غير محققتين مثلاً 3 من أجل $x > 10$ while (x > 10) و منه: كل loop نستعيض عنها بحسب الوقت إما بمسار واحد أو 3 مسارات أو 5 مسارات و كل path مضروب بعدد التفرعات.

- مثلاً في حال كان المجال: x ينتمي $[3, 10]$ اجتماع $[20, 30]$ سيكون:

- x محققة و منه $x=7$.
 - x قيم حدية مقبولة و منه $x = 3, x = 10, x = 20, x = 29$.
 - x قيم حدية مرفوضة و منه $x = 2, x = 11, x = 19, x = 30$.
- في الـ nested loop :

1. نختار نقطة نستطيع منها الدخول لحلقة داخلية و نقوم بما قمنا به للـ loop
2. فحص الحلقة الخارجية يؤخذ بعين الاعتبار أن قيمها تؤخذ بتكرار الـ loop الداخلية

3. Data flow testing (Definition use)

تحدد مسارات إختبار البرنامج وفقاً لمواقع التعاريف واستخدامات المتغيرات في البرنامج أي دراسة حركة تدفق البيانات لإيجاد testing path .

نعلم أن مالك الحلقة هو الشرط و من يتحكم بالشرط هو قيمة المتغير x ضمنه، و x هي قيمة ناتجة عن flow أي سلسلة تعليمات وليست متحكمة من قبل المستخدم، فإذا ركزنا على الـ statements المتحكمة بقيمة x ستكون هذه الوصلات المتعلقة فقط بـ x هي وصلات الـ definition use، أي أولاً نقوم بتأطير أرقام الـ statement التي سببت بتعريف المتغير x وما هي النقاط التي استخدمت المتغير x .

Definition use: ملاحقة الـ variable أين تم تعريفه و أين يتم استخدامه (أي متابعة التغيرات التي طرأت عليه من لحظة تعريفه لاستخدامه).

ماذا يعني use definition?

إن عملية التعريف هي عملية الكتابة وعملية الاستخدام هي عملية القراءة.
حالة القراءة (use) لها نوعان:

1. computation use (c-use): أي قيمة المتغير تستخدم ضمن عمليات حسابية ($x=x+1$) لحساب متغيرات أخرى أو حساب الخرج.
2. predicate use (p-use): أي قيمة المتغير تستخدم ضمن عملية منطقية ($x>0$) لتحديد مجرى (flow) البرنامج.

قمنا بهذا التفصيل لأننا نريد الوصول بتسلسلات c-use للمتغير نفسه إلى مرحلة p-use لمعرفة تماماً ما هي القيمة التي ستوصلنا لقيمة محققة أو غير محققة.

مثال: تحديد مواقع الـ Definition والـ Use لكل متغير:

ملاحظة: تحديد المواقع يكون بتحديد رقم السطر الذي يوجد فيه هذا المتغير.

```
1. input (x, y);
2. x=x+1;
3. y=y+1;
4. if (x>0)
5.     y=y+1;
   else
6.     y=y+2;
7. output (x, y);
```

$x=x+1$ هي عبارة عن definition و use في ان واحد أي كل شيء على اليسار هو Def و كل شيء على اليمين هو Use.

الحل:

يكون تحديد الثنائيات (def, use) بالنسبة لـ x, y كالتالي :

■ $x: (1, 2)$

هنا تم تعريف x بالسطر رقم 1 وأول استخدام للمتغير x كان بالسطر 2

■ $x: (2, 4 - 5)$

هنا ثاني تعريف لـ x هو في السطر 2 وكذلك تم استخدامه بهذا السطر ولكن تم ذكرها بـ (1,2) فلذلك ثاني استخدام هو بالسطر الرابع والخامس

■ $x: (2, 4 - 6)$

نلاحظ أن x تؤثر على السطر 4 و 6 ولذلك يجب ذكرها

■ $x: (2,7)$

وكذلك الأمر بالنسبة للمتغير y :

■ $y: (1, 3)$

■ $y: (3, 5), (3, 6)$

■ $y: (5,7), (6,7)$

ثم نقوم بإيجاد الـ **Testing path** لتغطيه الثنائيات السابقة ثم نوجد الـ **test case** المقابلة للـ **test path**

بناء على تعريف هذه الوصلات أصبحنا نعلم أن التحكم بالعقدة 2 (المسؤولة عن تعريف قيمة جديدة لـ x) على أساسها سيتم التحكم بالشرط في العقدة 4 بقبول الشرط أو رفضه أي إذا تم أخذ المسار 123457 الذي سيتحكم بالانتقال في هذا المسار هو قيمة x .

• Here the definitions are:

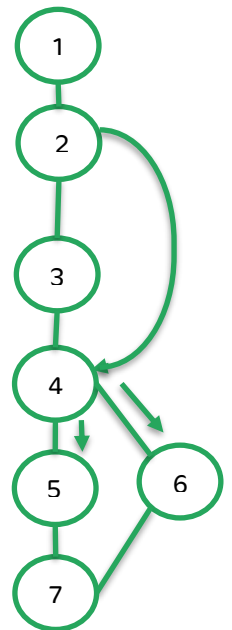
- 1,2 for x
- 1,3, 5 and 6 for y

• The c-uses are:

- 2 for x
- 3, 5, and 6 for y
- 7 for x and y

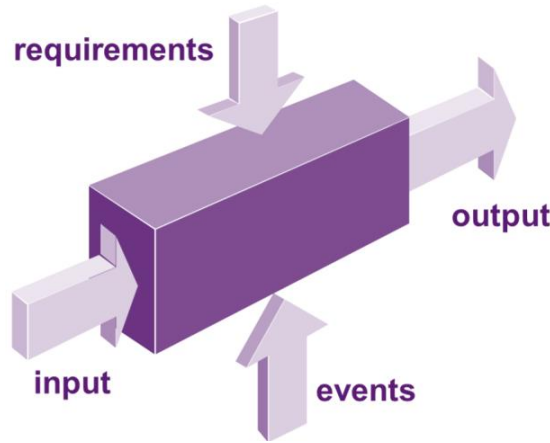
• The p-uses are:

- 4-5 and 4-6 for x



Black Box Testing

و هو فحص وظائف (functionality) البرنامج من وجهة نظر المستخدم بطريقة منضبطة وغير عشوائية.



1. Smoke testing:

المفاصل الأساسية major functionality نتأكد أن الأخطاء الخاصة بها تمت إزالتها، لا نفحص كل البرنامج وإنما فقط المفاصل الأساسية.

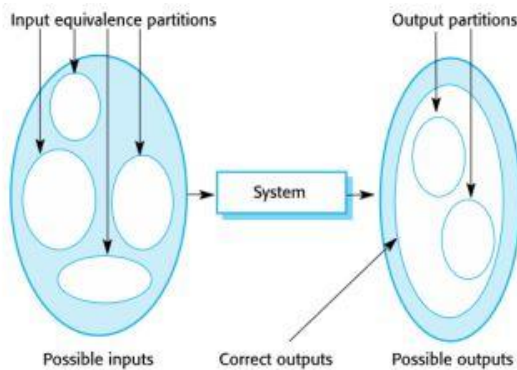
2. Use traceability testing

تستخدم للتحقق أن جميع متطلبات النظام تم تنفيذها بشكل صحيح ويمكن تتبعها طوال دورة حياة التطوير. وتتضمن ربط المتطلبات بأنشطة التصميم والتطوير والاختبار والنشر والتأكد من تنفيذ كل مطلب.

3. UI testing

هو الاختبار عن طريق الواجهات، ويتم التمثيل عن طريق مجموعة من nodes كل node لها دخل و يمكن أن تعطي خرج إما بنفس node أو في node آخر.
Node: هي عبارة عن وحدة عرض قد تكون عبارة عن page وحيدة أو مجموعة صفحات pages.
لحل مشكلة وجود احتمالات data input كبيرة و مستويات تغطية مختلفة سنلجأ إلى:

Equivalence partitioning:



التجزئة المتكافئة هي عملية تقسيم متكافئ للـ input domain لأجزاء وكل جزء هو مجموعة inputs تؤدي لنفس قيمة output ونقوم بذلك لنحدد من عملية الاختبار لعملية واحدة فقط، وذلك لنقطة عشوائية داخل هذا المجال وبالتالي نفحص عينة من كل مجموعة.

وشروط اختيار هذه العينة هي تحليل الحدود boundary analysis وتتم بنفس الطريقة التي استخدمناها مع loops الـ:

1. قيمة محققة حتماً في منتصف المجال
2. قيمة حدية محققة (عند طرف المجال)
3. قيمة غير محققة (خارج المجال)

■ اختيار الـ in test cases والـ functional test cases اعتماداً على قاعدة boundary analysis يعني تحليل ما هي حدود هذه المجموعة المتكافئة واختيار الداتا على حدودها المقبولة والمرفوضة.

selenium: UI automation functional testing

(سيلينيوم) عبارة عن أداة تسمح للمطورين بإنشاء و تشغيل اختبارات الوظائف والتحكم بواجهة المستخدم بشكل تلقائي حيث يولد مسارات الربط ما بين العقد، ويعطي عدد هائلاً من العقد، نقوم بكتابة script للتحكم بهذه العقد ودخلها ومن ثم نقوم بعمل run، فيتم عمل مسح scan لهذه العقد و التي تمثل واجهات ورسائل.

Non-functional testing:

1. Performance:

- الأداء من جميع النواحي مثل:

- Responding time
- Bandwidth conception
- استهلاك الـ Ram
- استهلاك الذاكرة

و لكن مع تحديد الحد المقبول لكل مما ورد سابقاً عملية الاختبار تكون متعلقة بالـ software و الـ hardware و apps التي تعمل على نفس الـ server.

2. Load testing

تكون من خلال :

- Number of users: عدد المستخدمين المتصلين في التطبيق في الوقت ذاته
- Data flow: كمية البيانات الواردة بالثانية

3. Stress testing

يختبر قدرة البرنامج على احتمال الضغط الشديد من المستخدمين أو الطلبات في نفس الوقت.

4. Reliability Testing

يهدف إلى تحديد مدى قدرة المنتج على العمل بشكل مستمر وموثوق به لفترات طويلة من الزمن.

- The End -

